

UNIT-II

RELATIONAL ALGEBRA:

- Relational algebra is a procedural query language, which takes instances of relations as input and yields instances of relations as output.
- It uses operators to perform queries. An operator can be either **unary** or **binary**. They accept relations as their input and yield relations as their output.
- Relational algebra is performed recursively on a relation and intermediate results are also considered relations.

Terminology

- Relation – a set of tuples.
- Tuple – a collection of attributes which describe some real world entity.
- Attribute – a real world role played by a named domain.
- Domain – a set of atomic values.
- Set – a mathematical definition for a collection of objects which contains no duplicates.

Unary operations: Operates on one relation. These include:

- **Selection**
- **Projection**
- **Renaming**

Selection Operation (σ):

It selects tuples that satisfy the given predicate from a relation.

Notation – $\sigma_p(r)$

Where σ stands for selection predicate and r stands for relation. p is propositional logic formula which may use connectors like **and**, **or**, and **not**. These terms may use relational operators like $=$, $\neq$, $\geq$, $<$, $>$, $\leq$.

For example –

$\sigma_{\text{subject} = \text{"database"}}(\text{Books})$

Output – Selects tuples from books where subject is 'database'.

$\sigma_{\text{subject} = \text{"database"} \text{ and } \text{price} = \text{"450"}}(\text{Books})$

Output – Selects tuples from books where subject is 'database' and 'price' is 450.

$\sigma_{\text{subject} = \text{"database"} \text{ and price} = \text{"450"} \text{ or year} > \text{"2010"}}(\text{Books})$

Output – Selects tuples from books where subject is 'database' and 'price' is 450 or those books published after 2010.

Project Operation (Π):

It projects column(s) that satisfy a given predicate.

Notation – $\Pi_{A_1, A_2, A_n}(r)$ Where A_1, A_2, A_n are attribute names of relation **r**.

Duplicate rows are automatically eliminated, as relation is a set.

For example –

$\Pi_{\text{subject, author}}(\text{Books})$

Selects and projects columns named as subject and author from the relation Books.

Rename Operation (ρ):

The results of relational algebra are also relations but without any name. The rename operation allows us to rename the output relation. 'rename' operation is denoted with small Greek letter **rho** ρ .

Notation – $\rho_x(E)$

Where the result of expression **E** is saved with name of **x**.

Binary operations: Operates on pairs of relation. These include:

- Union
- Set Difference
- Cartesian product
- Division
- Joins

Union Operation ($\cup$):

It performs binary union between two given relations and is defined as –

$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$

Notation –: $r \cup s$

Where **r** and **s** are either database relations or relation result set (temporary relation).

For a union operation to be valid, the following conditions must hold –

- **r**, and **s** must have the same number of attributes.
- Attribute domains must be compatible.
- Duplicate tuples are automatically eliminated.

Set Difference ($-$):

The result of set difference operation is tuples, which are present in one relation but are not in the second relation.

Notation –: ($r - s$)

Finds all the tuples that are present in **r** but not in **s**.

$\Pi_{\text{author}}(\text{Books}) - \Pi_{\text{author}}(\text{Articles})$

Output – Provides the name of authors who have written books but not articles.

Cartesian Product (X):

Combines information of two different relations into one.

Notation – $r \times s$

Where **r** and **s** are relations and their output will be defined as –

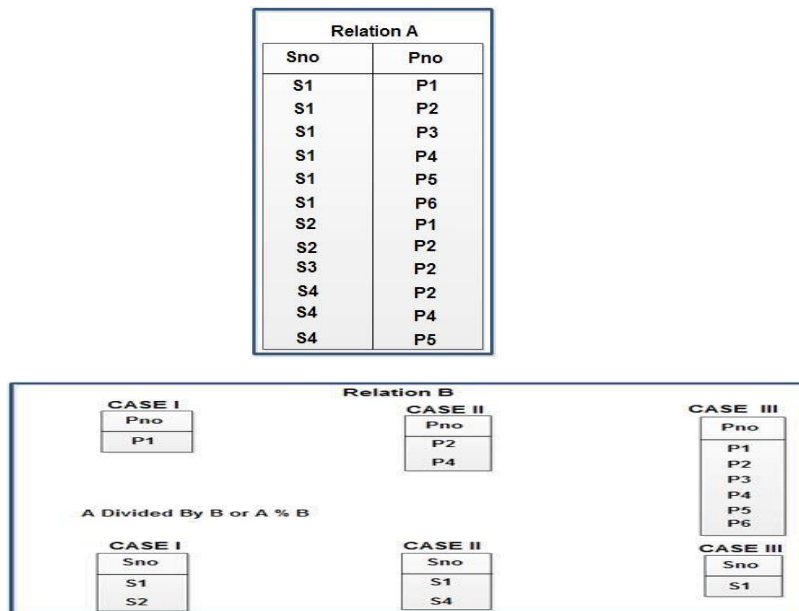
$$r \times s = \{ q \ t \mid q \in r \text{ and } t \in s \}$$

$\sigma_{\text{author} = \text{'tutorial'}}(\text{Books} \times \text{Articles})$

Output – Yields a relation, which shows all the books and articles written by tutorial.

Division:

The division operator divides a dividend relation A of degree (means number of columns in a relation) $m + n$ by a divisor relation B of degree n , and produces a resultant relation of degree m . It is denoted by $\div$.



In this example dividend relation A has two attributes of Sno, Pno (of degree 2) and division relation B has only one attribute Pno (of degree 1). Then, A divide by B gives a resultant relation of degree 1. It means it has only one attribute of Sno.

$$A = \text{SNO} * \text{SNO} = \text{SNO}. \quad B = \text{PNO}$$

The resultant relation has those tuples that are common values of those attributes, which appears in the resultant attribute sno. *For example, in CASE II,*

P2 has Snos. -- S1, S2, S3,S4

P4 has Snos. -- S1, S4

S1, S4 are the common supplier who supply both P2 and P4. So the resultant relation has tuples S1 and S4.

In CASE III : There is only one supplier S1 who supply all the parts from P1 to P6.

Joins:

We understand the benefits of taking a Cartesian product of two relations, which gives us all the possible tuples that are paired together. But it might not be feasible for us in certain cases to take a Cartesian product where we encounter huge relations with thousands of tuples having a considerable large number of attributes.

Join is a combination of a Cartesian product followed by a selection process. A Join operation pairs two tuples from different relations, if and only if a given join condition is satisfied.

We will briefly describe various join types in the following sections.

Theta(θ)Join:

Theta join combines tuples from different relations provided they satisfy the theta condition.

The join condition is denoted by the symbol θ .

Notation

$$R1 \bowtie_{\theta} R2$$

R1 and R2 are relations having attributes (A1, A2, .., An) and (B1, B2,.. ,Bn) such that the attributes don't have anything in common, that is $R1 \cap R2 = \Phi$.

Theta join can use all kinds of comparison operators.

Student		
SID	Name	Std
101	Alex	10
102	Maria	11

Subjects	
Class	Subject
10	Math
10	English
11	Music
11	Sports

Student_Detail –

$$STUDENT \bowtie_{Student.Std = Subject.Class} SUBJECT$$

Student_detail				
SID	Name	Std	Class	Subject
101	Alex	10	10	Math
101	Alex	10	10	English
102	Maria	11	11	Music
102	Maria	11	11	Sports

Equijoin:

When Theta join uses only **equality** comparison operator, it is said to be equijoin. The above example corresponds to equijoin.

Natural Join ($\bowtie$):

Natural join does not use any comparison operator. It does not concatenate the way a Cartesian product does. We can perform a Natural Join only if there is at least one common attribute that exists between two relations. In addition, the attributes must have the same name and domain.

Natural join acts on those matching attributes where the values of attributes in both the relations are same.

Courses		
CID	Course	Dept
CS01	Database	CS
ME01	Mechanics	ME
EE01	Electronics	EE

HoD	
Dept	Head
CS	Alex
ME	Maya
EE	Mira

Courses $\bowtie$ HoD			
Dept	CID	Course	Head
CS	CS01	Database	Alex
ME	ME01	Mechanics	Maya
EE	EE01	Electronics	Mira

Outer Joins: Theta Join, Equijoin, and Natural Join are called inner joins. An inner join includes only those tuples with matching attributes and the rest are discarded in the resulting relation. Therefore, we need to use outer joins to include all the tuples from the participating relations in the resulting relation. There are three kinds of outer joins – left outer join, right outer join, and full outer join.

Left Outer Join ($R \bowtie^L S$):

All the tuples from the Left relation, R, are included in the resulting relation. If there are tuples in R without any matching tuple in the Right relation S, then the S-attributes of the resulting relation are made NULL.

Left	
A	B
100	Database
101	Mechanics
102	Electronics

Right	
A	B
100	Alex
102	Maya
104	Mira

Courses $\bowtie^L$ HoD			
A	B	C	D
100	Database	100	Alex
101	Mechanics	---	---
102	Electronics	102	Maya

Right Outer Join ($R \bowtie^R S$):

All the tuples from the Right relation, S, are included in the resulting relation. If there are tuples in S without any matching tuple in R, then the R-attributes of resulting relation are made NULL.

Courses $\bowtie^R$ HoD			
A	B	C	D
100	Database	100	Alex
102	Electronics	102	Maya
---	---	104	Mira

SN):

Courses		HoD	
A	B	C	D
100	Database	100	Alex
101	Mechanics	---	---
102	Electronics	102	Maya
---	---	104	Mira

Courses		HoD	
A	B	C	D
100	Database	100	Alex
101	Mechanics	---	---
102	Electronics	102	Maya
---	---	104	Mira

The main construct for representing data in the relational model is a **relation**. A relation consists of a **relation schema** and a **relation instance**. The relation instance is a table, and the relation schema describes the column heads for the table. We first describe the relation schema and then the relation instance. The schema species the relation's name, the name of each **field** (or **column**, or **attribute**) and the **domain** of each field. A domain is referred to in a relation schema by the **domain name** and has a set of associated **values**.

We use the example of student information:

Students (*sid*: string, *name*: string, *login*: string, *age*: integer, *gpa*:real)

This says, for instance, that the field named *sid* has a domain named string. The set of values associated with domain string is the set of all character strings. We now turn to the instances of a relation. An **instance** of a relation is a set of **tuples**, also called **records**, in which each tuple has the same number of fields as the relation schema. A relation instance can be thought of as a *table* in which each tuple is a *row*, and all rows have the same number of fields. (The term *relation instance* is often abbreviated to just *relation*, when there is no confusion with other aspects of a relation such as its schema.)

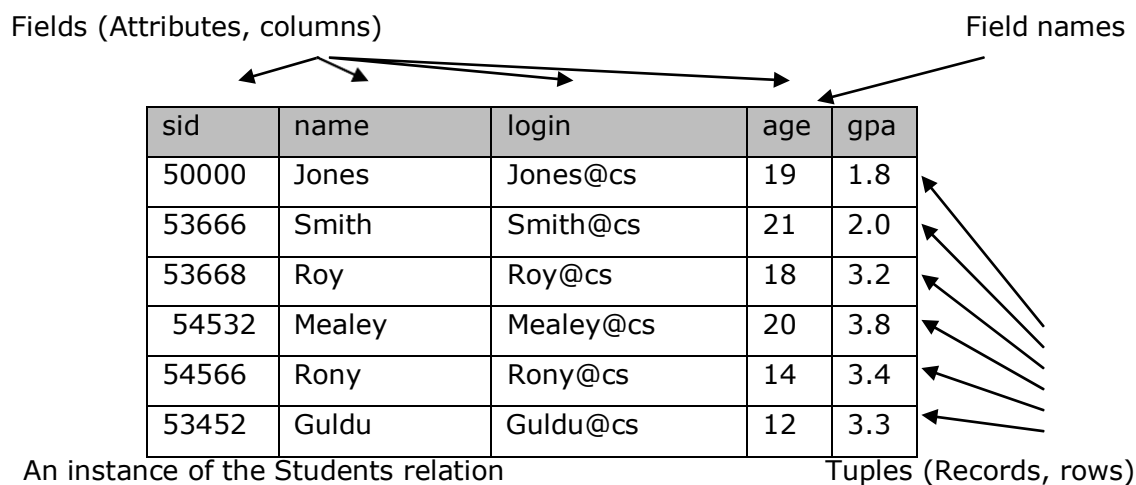


Table 2.1

The instance S1 contains six tuples and has, as we expect from the schema, five fields. Note that no two rows are identical. This is a requirement of the relational model each relation is defined to be a set of unique tuples or rows.¹ The order in which the rows are listed is not important. Thus sid is field 1 of Students, login is field 3, and so on. If this convention is used, the order of fields is significant. Most database systems use a combination of these

conventions. For example, in SQL the named fields convention is used in statements that retrieve tuples, and the ordered fields convention is commonly used when inserting tuples.

A relation schema specifies the domain of each field or column in the relation instance.

These **domain constraints** in the schema specify an important condition that we want each instance of the relation to satisfy: The values that appear in a column must be drawn from the domain associated with that column. Thus, the domain of a field is essentially the *type* of that field, in programming language terms, and restricts the values that can appear in the field.

The **degree**, also called **arity**, of a relation is the number of fields. The **cardinality** of a relation instance is the number of tuples in it. In Figure the degree of the relation (the number of columns) is five, and the cardinality of this instance is six. A **relational database** is a collection of relations with distinct relation names. The **relational database schema** is the collection of schemas for the relations in the database.

Relational Model

The relational model is today the primary data model for commercial data processing applications. It attained its primary position because of its simplicity, which eases the job of the programmer, compared to earlier data models such as the network model or the hierarchical model. In this, we first study the fundamentals of the relational model. A substantial theory exists for relational databases.

Structure of Relational Databases:

A relational database consists of a collection of tables, each of which is assigned a unique name. For example, consider the instructor table of Figure:1.5, which stores information about instructors. The table has four column headers: ID, name, dept name, and salary. Each row of this table records information about an instructor, consisting of the instructor's ID, name, dept name, and salary. Similarly, the course table of Figure 1.6 stores information about courses, consisting of a course id, title, dept name, and credits, for each course. Note that each instructor is identified by the value of the column ID, while each course is identified by the value of the column course id.

Figure 1.7 shows a third table, prereq, which stores the prerequisite courses for each course. The table has two columns, course id and prereq id. Each row consists of a pair of course identifiers such that the second course is a prerequisite for the first course.

Thus, a row in the prereq table indicates that two courses are related in the sense that one course is a prerequisite for the other. As another example, we consider the table instructor, a row in the table can be thought of as representing the relationship between a specified ID and the

corresponding values for name,dept name, and salary values.

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 1.5: The instructor relation (2.1)

In general, a row in a table represents a relationship among a set of values. Since a table is a collection of such relationships, there is a close correspondence between the concept of table and the mathematical concept of relation, from which the relational data model takes its name. In mathematical terminology, a tuple is simply a sequence (or list) of values. A relationship between n values is represented mathematically by an n -tuple of values, i.e., a tuple with n values, which corresponds to a row in a table.

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Figure: 1.6: The course relation (2.2)

course_id	prereq_id
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Figure: 1.7: The prereq relation. (2.3)

Thus, in the relational model the term relation is used to refer to a table, while the term tuple is used to refer to a row. Similarly, the term attribute refers to a column of a table.

Examining Figure 1.5, we can see that the relation instructor has four attributes:

ID, name, dept name, and salary.

We use the term relation instance to refer to a specific instance of a relation, i.e., containing a specific set of rows. The instance of instructor shown in Figure 1.5 has 12 tuples, corresponding to 12 instructors.

In this topic, we shall be using a number of different relations to illustrate the various concepts underlying the relational data model. These relations represent part of a university. They do not include all the data an actual university database would contain, in order to simplify our presentation.

The order in which tuples appear in a relation is irrelevant, since a relation is a set of tuples. Thus, whether the tuples of a relation are listed in sorted order, as in Figure 1.5, or are unsorted, as in Figure 1.8, does not matter; the relations in the two figures are the same, since both contain the same set of tuples. For ease of exposition, we will mostly show the relations sorted by their first attribute. For each attribute of a relation, there is a set of permitted values, called the domain of that attribute. Thus, the domain of the salary attribute of the instructor relation is the set of all possible salary values, while the domain of the name attribute is the set of all possible instructor names.

We require that, for all relations r , the domains of all attributes of r be atomic. A domain is atomic if elements of the domain are considered to be indivisible units.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Figure: 1.8: Unsorted display of the instructor relation. (2-4)

For example, suppose the table instructor had an attribute phone number, which can store a set of phone numbers corresponding to the instructor. Then the domain of phone number would not be atomic, since an element of the domain is a set of phone numbers, and it has subparts, namely the individual phone numbers in the set.

The important issue is not what the domain itself is, but rather how we use domain elements in our database. Suppose now that the phone number attribute stores a single phone number. Even

then, if we split the value from the phone number attribute into a country code, an area code and a local number, we would be treating it as a nonatomic value. If we treat each phone number as a single indivisible unit, then the attribute phone number would have an atomic domain.

The null value is a special value that signifies that the value is unknown or does not exist. For example, suppose as before that we include the attribute phone number in the instructor relation. It may be that an instructor does not have a phone number at all, or that the telephone number is unlisted. We would then have to use the null value to signify that the value is unknown or does not exist. We shall see later that null values cause a number of difficulties when we access or update the database, and thus should be eliminated if at all possible. We shall assume null values are absent initially.

Database Schema

When we talk about a database, we must differentiate between the database schema, which is the logical design of the database, and the database instance, which is a snapshot of the data in the database at a given

instant in time. The concept of a relation corresponds to the programming-language notion of a variable, while the concept of a relation schema corresponds to the programming-language notion of type definition.

In general, a relation schema consists of a list of attributes and their corresponding domains. The concept of a relation instance corresponds to the programming-language notion of a value of a variable. The value of a given variable may change with time;

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Figure 1.9: The department relation.(2-5)

similarly the contents of a relation instance may change with time as the relation is updated. In contrast, the schema of a relation does not generally change. Although it is important to know the difference between a relation schema and a relation instance, we often use the same name, such as instructor, to refer to both the schema and the instance. Where required, we explicitly refer to the schema or to the instance, for example "the instructor schema," or "an instance of the instructor relation." However, where it is clear whether we mean the schema or the instance, we simply use the relation name.

Consider the department relation of Figure 1.9. The schema for that relation is

department (dept name, building, budget)

Note that the attribute dept name appears in both the instructor schema and the department schema. This duplication is not a coincidence. Rather, using common attributes in relation schemas is one way of relating tuples of distinct relations.

For example, suppose we wish to find the information about all the instructors who work in the Watson building. We look first at the department relation to find the dept name of all the departments housed in Watson. Then, for each such department, we look in the instructor relation to find the information about the instructor associated with the corresponding dept name.

Let us continue with our university database example. Each course in a university may be offered multiple times, across different semesters, or even within a semester. We need a relation to describe each individual

offering, or section, of the class. The schema is

section (course id, sec id, semester, year, building, room number, time slot id)

Figure 1.10 shows a sample instance of the section relation. We need a relation to describe the association between instructors and the class sections that they teach. The relation schema to describe this association is

teaches (ID, course id, sec id, semester, year)

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Figure 1.10: The section relation.(2-6)

Figure 1.11 shows a sample instance of the teaches relation. As you can imagine, there are many more relations maintained in a real university database. In addition to those relations we have listed already, instructor, department, course, section, prereq, and teaches, we use the following relations in this text:

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Figure: 1.11: The teaches relation.(2-7)

student (ID, name, dept name, tot cred)

advisor (s id, i id)

takes (ID, course id, sec id, semester, year, grade)

classroom (building, room number, capacity)

Key time slot (time slot id, day, start time, end time)

s

We must have a way to specify how tuples within a given relation are distinguished. This is expressed in terms of their attributes. That is, the values of the attribute values of a tuple must be such that they can uniquely identify the tuple. In other words, no two tuples in a relation are allowed to have exactly the same value for all attributes.

A superkey is a set of one or more attributes that, taken collectively, allow us to identify uniquely a tuple in the relation. For example, the ID attribute of the relation instructor is sufficient to distinguish one instructor tuple from

another. Thus, ID is a superkey. The name attribute of instructor, on the other hand, is not a superkey, because several instructors might have the same name. Formally, let R denote the set of attributes in the schema of relation r . If we say that a subset K of R is a superkey for r , we are restricting consideration to instances of relations r in which no two distinct tuples have the same values on all attributes in K . That is, if t_1 and t_2 are in r and $t_1 = t_2$, then $t_1.K = t_2.K$.

A superkey may contain extraneous attributes. For example, the combination of ID and name is a superkey for the relation instructor. If K is a superkey, then so is any superset of K . We are often interested in superkeys for which no proper subset is a superkey. Such minimal superkeys are called candidate keys.

It is possible that several distinct sets of attributes could serve as a candidate key. Suppose that a combination of name and dept name is sufficient to distinguish among members of the instructor relation. Then, both $\{ID\}$ and $\{name, dept\}$ are candidate keys. Although the attributes ID and name together can distinguish instructor tuples, their combination, $\{ID, name\}$, does not form a candidate key, since the attribute ID alone is a candidate key.

We shall use the term primary key to denote a candidate key that is chosen by the database designer as the principal means of identifying tuples within a relation. A key (whether primary, candidate, or super) is a property of the entire relation, rather than of the individual tuples. Any two individual tuples in the relation are prohibited from having the same value on the key attributes at the same time. The designation of a key represents a constraint in the real-world enterprise being modeled.

Primary keys must be chosen with care. As we noted, the name of a person is obviously not sufficient, because there may be many people with the same name. In the United States, the social-security number attribute of a person would be a candidate key. Since non-U.S. residents usually do not have social-security numbers, international enterprises must generate their own unique identifiers.

An alternative is to use some unique combination of other attributes as a key. The primary key should be chosen such that its attribute values are never, or very rarely, changed. For instance, the address field of a person should not be part of the primary key, since it is likely to change. Social-security numbers, on the other hand, are guaranteed never to change. Unique identifiers generated by enterprises generally do not change, except if two enterprises merge; in such a case the same identifier may have been issued by both enterprises, and a reallocation of identifiers may be required to make sure they are unique.

It is customary to list the primary key attributes of a relation schema before the other attributes; for example, the dept name attribute of department is listed first, since it is the primary key. Primary key attributes are also underlined. A relation, say r_1 , may include among its attributes the primary key of another relation, say r_2 . This attribute is called a foreign key from r_1 ,

referencing r2.

The relation r1 is also called the referencing relation of the foreign key dependency, and r2 is called the referenced relation of the foreign key. For example, the attribute dept name in instructor is a foreign key from instructor, referencing department, since dept name is the primary key of department. In any database instance, given any tuple, say ta, from the instructor relation, there must be some tuple, say tb, in the department relation such that the value of the dept name attribute of ta is the same as the value of the primary key, dept name, of tb.

Now consider the section and teaches relations. It would be reasonable to require that if a section exists for a course, it must be taught by at least one instructor; however, it could possibly be taught by more than one instructor. To enforce this constraint, we would require that if a particular (course id, sec id, semester, year) combination appears in section, then the same combination must appear in teaches. However, this set of values does not form a primary key for teaches, since more than one instructor may teach one such section. As a result, we cannot declare a foreign key constraint from section to teaches (although we can define a foreign key constraint in the other direction, from teaches to section).

The constraint from section to teaches is an example of a referential integrity constraint; a referential integrity constraint requires that the values appearing in specified attributes of any tuple in the referencing relation also appear in specified attributes of at least one tuple in the referenced relation.

Schema Diagrams

A database schema, along with primary key and foreign key dependencies, can be depicted by schema diagrams. Figure 1.12 shows the schema diagram for our university organization. Each relation appears as a box, with the relation name at the top in blue, and the attributes listed inside the box. Primary key attributes are shown underlined. Foreign key dependencies appear as arrows from the foreign key attributes of the referencing relation to the primary key of the referenced relation.

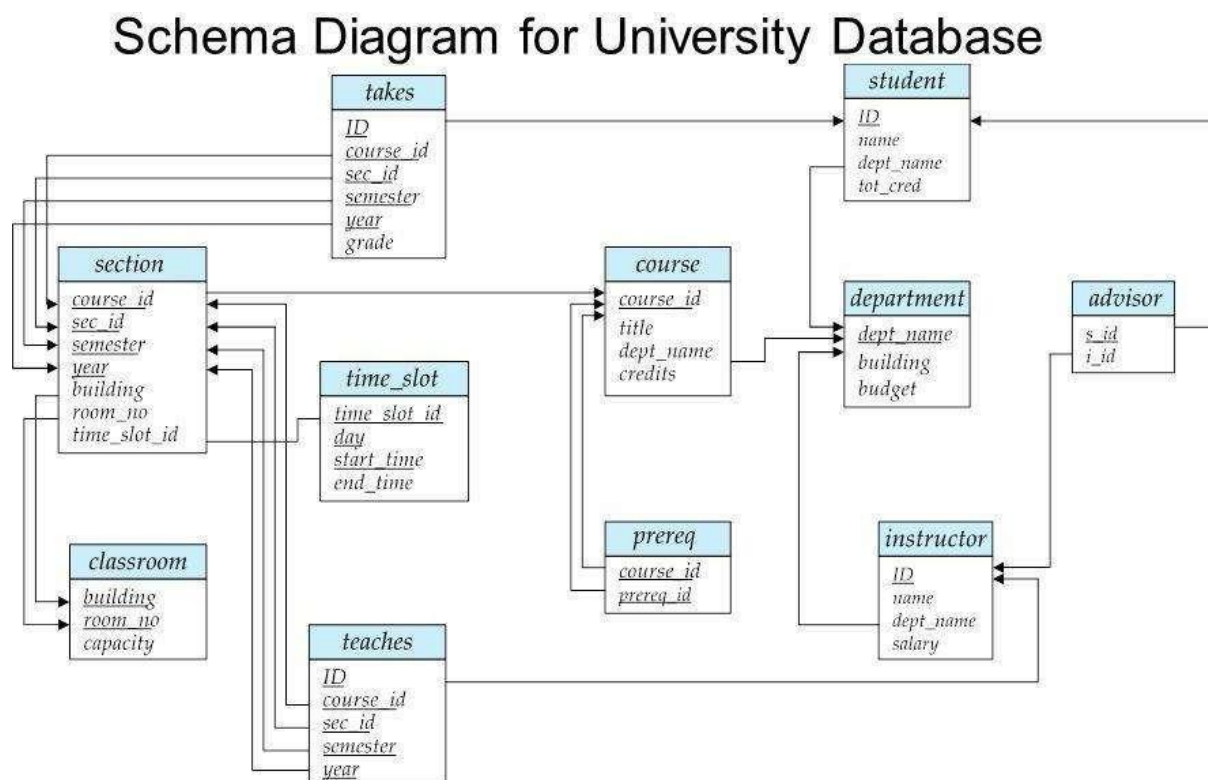
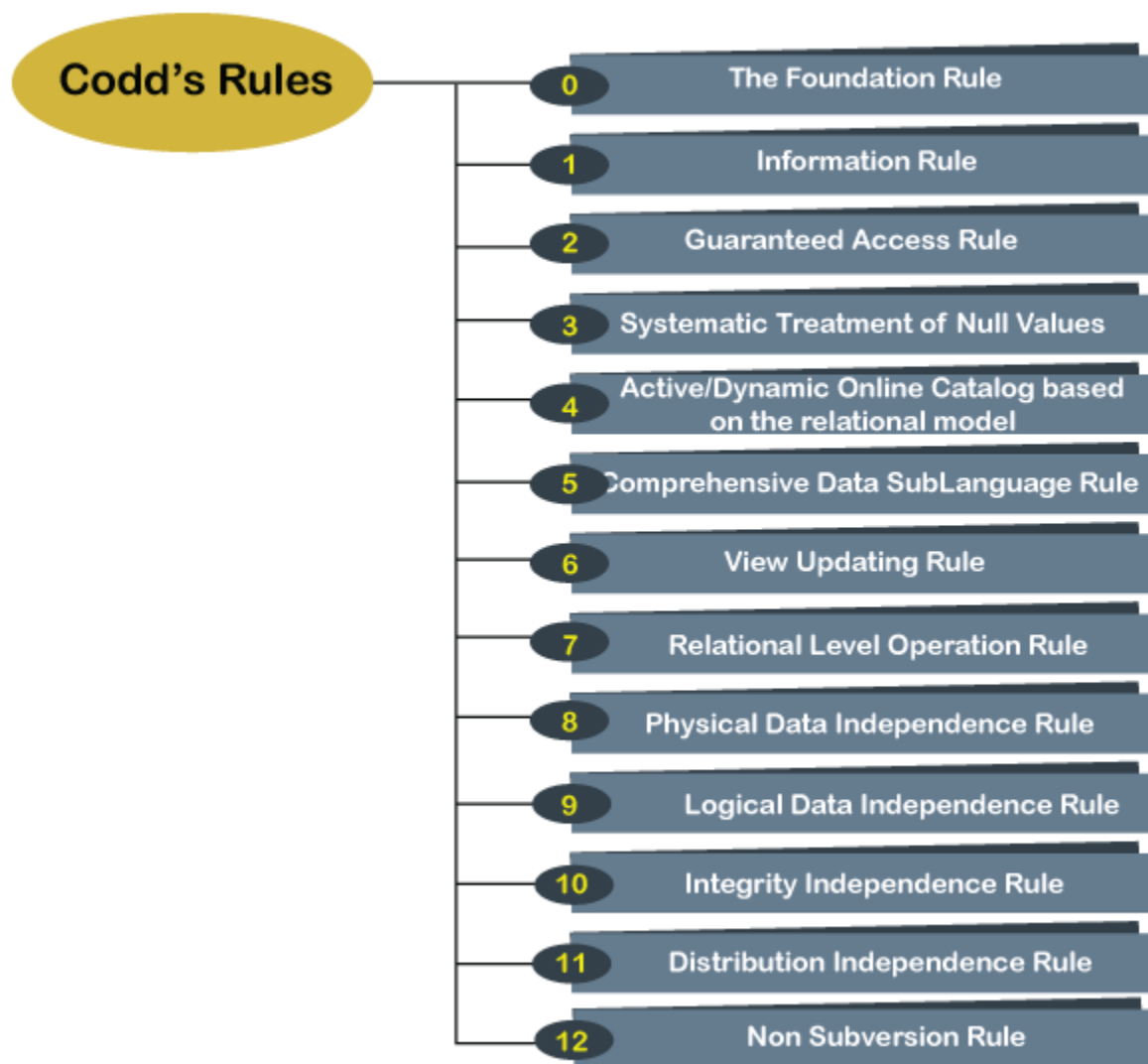


Figure 1.12 : Schema diagram for the university database.

Referential integrity constraints other than foreign key constraints are not shown explicitly in schema diagrams. We will study a different diagrammatic representation called the entity-relationship diagram.

12 Codd's Rules

Every database has tables, and constraints cannot be referred to as a rational database system. And if any database has only relational data model, it cannot be a **Relational Database System (RDBMS)**. So, some rules define a database to be the correct RDBMS. These rules were developed by **Dr. Edgar F. Codd (E.F. Codd)** in **1985**, who has vast research knowledge on the Relational Model of database Systems. Codd presents his 13 rules for a database to test the concept of **DBMS** against his relational model, and if a database follows the rule, it is called a **true relational database (RDBMS)**. These 13 rules are popular in RDBMS, known as **Codd's 12 rules**.



Rule 0: The Foundation Rule

The database must be in relational form. So that the system can handle the database through its relational capabilities.

Rule 1: Information Rule

A database contains various information, and this information must be stored in each cell of a table in the form of rows and columns.

Rule 2: Guaranteed Access Rule

Every single or precise data (atomic value) may be accessed logically from a relational database using the combination of primary key value, table name, and column name.

Rule 3: Systematic Treatment of Null Values

This rule defines the systematic treatment of Null values in database records. The null value has various meanings in the database, like missing the data, no value in a cell, inappropriate information, unknown data and the primary key should not be null.

Rule 4: Active/Dynamic Online Catalog based on the relational model

It represents the entire logical structure of the descriptive database that must be stored online and is known as a database dictionary. It authorizes users to access the database and implement a similar query language to access the database.

Rule 5: Comprehensive Data SubLanguage Rule

The relational database supports various languages, and if we want to access the database, the language must be the explicit, linear or well-defined syntax, character strings and supports the comprehensive: data definition, view definition, data manipulation, integrity constraints, and limit transaction management operations. If the database allows access to the data without any language, it is considered a violation of the database.

Rule 6: View Updating Rule

All views table can be theoretically updated and must be practically updated by the database systems.

Rule 7: Relational Level Operation (High-Level Insert, Update and delete) Rule

A database system should follow high-level relational operations such as insert, update, and delete in each level or a single row. It also supports union, intersection and minus operation in the database system.

Rule 8: Physical Data Independence Rule

All stored data in a database or an application must be physically independent to access the database. Each data should not depend on other data or an application. If data is updated or the physical structure of the database is changed, it will not show any effect on external applications that are accessing the data from the database.

Rule 9: Logical Data Independence Rule

It is similar to physical data independence. It means, if any changes occurred to the logical level (table structures), it should not affect the user's view (application). For example, suppose a table either split into two tables, or two table joins to create a single table, these changes should not be impacted on the user view application.

Rule 10: Integrity Independence Rule

A database must maintain integrity independence when inserting data into table's cells using the SQL query language. All entered values should not be changed or rely on any external factor or application to maintain integrity. It is also helpful in making the database-independent for each front-end application.

Rule 11: Distribution Independence Rule

The distribution independence rule represents a database that must work properly, even if it is stored in different locations and used by different end-users. Suppose a user accesses the database through an application; in that case, they should not be aware that another user uses particular data, and the data they always get is only located on one site. The end users can access the database, and these access data should be independent for every user to perform the SQL queries.

Rule 12: Non Subversion Rule

The non-submersion rule defines RDBMS as a [SQL](#) language to store and manipulate the data in the database. If a system has a low-level or separate language other than SQL to access the database system, it should not subvert or bypass integrity to transform data.

What is SQL?

- SQL stands for Structured Query Language. It is used for storing and managing data in Relational Database Management System (RDBMS).
- It is a standard language for Relational Database System. It enables a user to create, read, update and delete relational databases and tables.
- All the RDBMS like MySQL, Informix, Oracle, MS Access and SQL Server use SQL as their standard database language.
- SQL allows users to query the database in a number of ways, using English-like statements.

SQL follows the following rules:

- Structure query language is not case sensitive. Generally, keywords of SQL are written in uppercase.

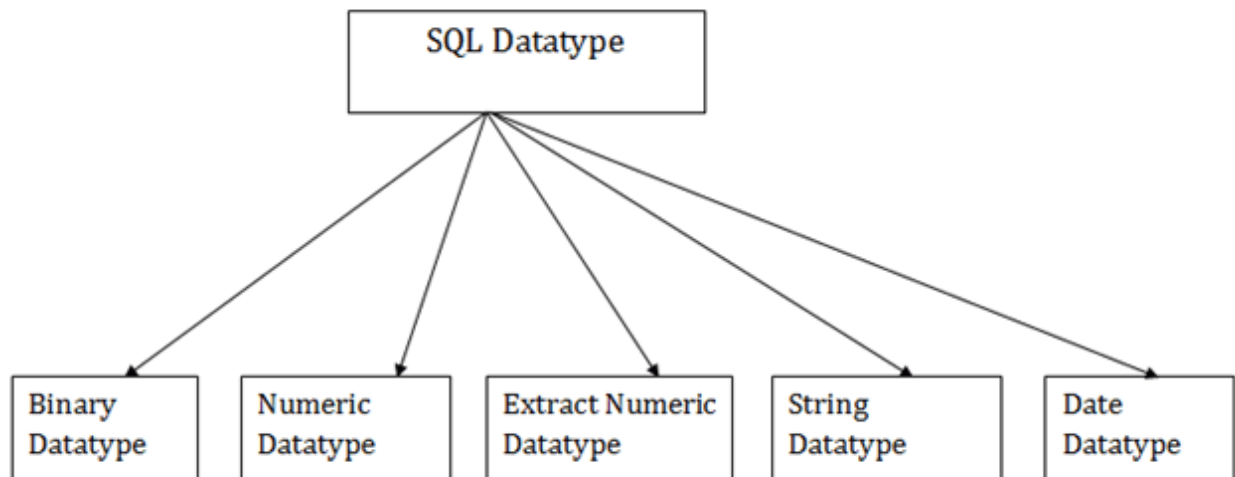
- Statements of SQL are dependent on text lines. We can use a single SQL statement on one or multiple text line.
- Using the SQL statements, you can perform most of the actions in a database.
- SQL depends on tuple relational calculus and relational algebra
- Statements of SQL are dependent on text lines. We can use a single SQL statement on one or multiple text line.
- Using the SQL statements, you can perform most of the actions in a database.
- SQL depends on tuple relational calculus and relational algebra.

What is SQL Process?

- When an SQL command is executing for any RDBMS, then the system figure out the best way to carry out the request and the SQL engine determines that how to interpret the task.
- In the process, various components are included. These components can be optimization Engine, Query engine, Query dispatcher, classic, etc.
- All the non-SQL queries are handled by the classic query engine, but SQL query engine won't handle logical files.
- When an SQL command is executing for any RDBMS, then the system figure out the best way to carry out the request and the SQL engine determines that how to interpret the task.
- In the process, various components are included. These components can be optimization Engine, Query engine, Query dispatcher, classic, etc.
- All the non-SQL queries are handled by the classic query engine, but SQL query engine won't handle logical files. What is SQL Process? What is Advantages of SQL?
- High speed
- No coding needed
- Well defined standards
- Portability
- Interactive language
- Multiple data view

What is SQL Datatype?

- SQL Datatype is used to define the values that a column can contain.
- Every column is required to have a name and data type in the database table. SQL Commands
- SQL commands are instructions. It is used to communicate with the database. It is also used to perform specific tasks, functions, and queries of data.
- SQL can perform various tasks like create a table, add data to tables, drop the table, modify the table, set permission for users.
- SQL commands are instructions. It is used to communicate with the database. It is also used to perform specific tasks, functions, and queries of data.
- SQL can perform various tasks like create a table, add data to tables, drop the table, modify the table, set permission for users.



1. Binary Datatypes

Data Type	Description
binary	It has a maximum length of 8000 bytes. It contains fixed-length binary data.
varbinary	It has a maximum length of 8000 bytes. It contains variable-length binary data.
image	It has a maximum length of 2,147,483,647 bytes. It contains variable-length binary data.

2. Approximate Numeric Datatype :

The subtypes are given below:

Data type	From	To	Description
float	-1.79E + 308	1.79E + 308	It is used to specify a floating-point value e.g. 6.2, 2.9 etc.
real	-3.40e + 38	3.40E + 38	It specifies a single precision floating point number

3. Exact Numeric Datatype

The subtypes are given below:

Data type	Description
int	It is used to specify an integer value.
smallint	It is used to specify small integer value.
bit	It has the number of bits to store.
decimal	It specifies a numeric value that can have a decimal number.
numeric	It is used to specify a numeric value.

4. Character String Datatype

The subtypes are given below:

Data type	Description
char	It has a maximum length of 8000 characters. It contains Fixed-length non-unicode characters.
varchar	It has a maximum length of 8000 characters. It contains variable-length non-unicode characters.
text	It has a maximum length of 2,147,483,647 characters. It contains variable-length non-unicode characters.

5. Date and time Datatypes

The subtypes are given below:

Datatype	Description
date	It is used to store the year, month, and days value.
time	It is used to store the hour, minute, and second values.
timestamp	It stores the year, month, day, hour, minute, and the second value.

Specifying Constraints:

Constraints in SQL means we are applying certain conditions or restrictions on the database. This further means that before inserting data into the database, we are checking for some conditions. If the condition we have applied to the database holds true for the data which is to be inserted, then only the data will be inserted into the database tables.

NOT NULL
UNIQUE
PRIMARY KEY
FOREIGN KEY
CHECK
DEFAULT
CREATE INDEX

NOT NULL

- NULL means empty, i.e., the value is not available.
- Whenever a table's column is declared as NOT NULL, then the value for that column cannot be empty for any of the table's records.
- There must exist a value in the column to which the NOT NULL constraint is applied.

Syntax :

```
CREATE TABLE TableName (ColumnName1 datatype NOT NULL, ColumnName2 datatype, ..., ColumnNameN datatype);
```

Example :

```
CREATE TABLE student(StudentID INT NOT NULL, Student_FirstName VARCHAR(20), Student_LastName VARCHAR(20), Student_PhoneNumber VARCHAR(20), Student_Email_ID VARCHAR(40));
```

```
ALTER TABLE TableName CHANGE Old_ColumnName New_ColumnName Datatype NOT NULL;
```

UNIQUE

- Duplicate values are not allowed in the columns to which the UNIQUE constraint is applied.
- The column with the unique constraint will always contain a unique value.
- This constraint can be applied to one or more than one column of a table, which means more than one unique constraint can exist on a single table.
- Using the UNIQUE constraint, you can also modify the already created tables.

Syntax:

```
CREATE TABLE TableName (ColumnName1 datatype UNIQUE, ColumnName2 datatype, ..., ColumnNameN datatype);
```

Example:

Create a student table and apply a UNIQUE constraint on one of the table's column while creating a table.

```
CREATE TABLE student(StudentID INT UNIQUE, Student_FirstName VARCHAR(20), Student_LastName VARCHAR(20), Student_PhoneNumber VARCHAR(20), Student_Email_ID VARCHAR(40));
```

PRIMARY KEY

- PRIMARY KEY Constraint is a combination of NOT NULL and Unique constraints.
- NOT NULL constraint and a UNIQUE constraint together forms a PRIMARY constraint.
- The column to which we have applied the primary constraint will always contain a unique value and will not allow null values.

Syntax:

```
CREATE TABLE TableName (ColumnName1 datatype PRIMARY KEY, ColumnName2 datatype, ..., ColumnNameN datatype);
```

Example:

Create a student table and apply the PRIMARY KEY constraint while creating a table.

```
CREATE TABLE student(StudentID INT PRIMARY KEY, Student_FirstName VARCHAR(20), Student_LastName VARCHAR(20), Student_PhoneNumber VARCHAR(20), Student_Email_ID VARCHAR(40));
```

FOREIGN KEY

- A foreign key is used for referential integrity.
- When we have two tables, and one table takes reference from another table, i.e., the same column is present in both the tables and that column acts as a primary key in one table. That particular column will act as a foreign key in another table.

Syntax:

```
CREATE TABLE tablename(Column1 Datatype(SIZE) PRIMARY KEY, ColumnNameN Datatype(SIZE), FOREIGN KEY(ColumnName) REFERENCES PARENT_TABLE_NAME(Primary_Key_ColumnName));
```

Example:

Create an employee table and apply the FOREIGN KEY constraint while creating a table.

To create a foreign key on any table, first, we need to create a primary key on a table.

```
CREATE TABLE employee (Emp_ID INT NOT NULL PRIMARY KEY, Emp_Name VARCHAR (40), Emp_Salary VARCHAR (40));
```

We will write a query to apply a foreign key on the department table referring to the primary key of the employee table, i.e., Emp_ID.

```
CREATE TABLE department(Dept_ID INT NOT NULL PRIMARY KEY, Dept_Name VARCHAR(40), Emp_ID INT NOT NULL, FOREIGN KEY(Emp_ID) REFERENCES employee(Emp_ID));
```

Syntax to apply the foreign key constraint with constraint name:

```
CREATE TABLE tablename(Column1 Datatype PRIMARY KEY, ColumnNameN Datatype(SIZE), CONSTRAINT ConstraintName FOREIGN KEY( ColumnName ) REFERENCES PARENT_TABLE_NAME(Primary_Key_ColumnName));
```

Example:

Create an employee table and apply the FOREIGN KEY constraint with a constraint name while creating a table.

To create a foreign key on any table, first, we need to create a primary key on a table.

```
CREATE TABLE employee (Emp_ID INT NOT NULL PRIMARY KEY, Emp_Name VARCHAR (40), Emp_Salary VARCHAR (40));
```

Now, we will write a query to apply a foreign key with a constraint name on the department table referring to the primary key of the employee table, i.e., Emp_ID.

```
CREATE TABLE department(Dept_ID INT NOT NULL PRIMARY KEY, Dept_Name VARCHAR(40), Emp_ID INT NOT NULL, CONSTRAINT emp_id_fk FOREIGN KEY(Emp_ID) REFERENCES employee(Emp_ID));
```

CHECK constraint

In SQL, a CHECK constraint is used to specify a condition that must be true for each row in a table. This condition is evaluated for each row at the time of insertion or update, and if the condition is not satisfied, the operation is rejected. The CHECK constraint is often used to enforce domain integrity by restricting the values that can be inserted into a column.

Here is the basic syntax for adding a CHECK constraint to a table:

Syntax

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    -- other columns,  
    CONSTRAINT constraint_name CHECK (condition)  
);
```

Example :

```
CREATE TABLE employees (
```

```
employee_id INT PRIMARY KEY,  
  
employee_name VARCHAR(50),  
  
salary DECIMAL(10, 2),  
  
CONSTRAINT check_salary CHECK (salary >= 0)  
  
);
```

DEFAULT constraint

DEFAULT constraint is used to provide a default value for a column when no value is explicitly specified during an INSERT operation. The default value is applied only if the column is not explicitly mentioned in the INSERT statement or if the INSERT statement does not provide a value for that particular column

Here is the basic syntax for adding a DEFAULT constraint to a column:

```
CREATE TABLE table_name (  
  
    column1 datatype DEFAULT default_value,  
  
    column2 datatype,  
  
    -- other columns  
  
);
```

Example

```
CREATE TABLE products (  
  
    product_id INT PRIMARY KEY,  
  
    product_name VARCHAR(50) NOT NULL,  
  
    price DECIMAL(10, 2) DEFAULT 0.00,  
  
    quantity_in_stock INT DEFAULT 0  
  
);
```

In this example:

The price column has a default value of 0.00. If you don't specify a value for price during an INSERT operation, it will be set to 0.00 by default.

The quantity_in_stock column has a default value of 0. If you don't specify a value for quantity_in_stock during an INSERT operation, it will be set to 0 by default.

You can also add a DEFAULT constraint to an existing column using the ALTER TABLE statement. For example:

```
ALTER TABLE products
```

```
ALTER COLUMN price SET DEFAULT 0.00;
```

This statement alters the price column in the products table, setting its default value to 0.00.

CREATE INDEX statement is used to create an index on one or more columns of a table. An index is a database object that improves the speed of data retrieval operations on a database table at the cost of additional writes and storage space. Indexes are particularly useful for speeding up queries that involve the columns on which the index is created.

Here is the basic syntax for creating an index on one column:

```
CREATE INDEX index_name
```

```
ON table_name (column_name);
```

And for creating an index on multiple columns:

```
CREATE INDEX index_name
```

```
ON table_name (column1, column2, ...);
```

Let's look at an example:

```
CREATE TABLE employees (
```

```
    employee_id INT PRIMARY KEY,
```

```
    employee_name VARCHAR(50),
```

```
    department_id INT,
```

```
    salary DECIMAL(10, 2)
```

);

- Creating an index on the department_id column

```
CREATE INDEX idx_department ON employees (department_id);
```

In this example, an index named idx_department is created on the department_id column of the employees table. This index will improve the speed of queries that involve the department_id column.

Basic Retrieval Queries in SQL

In a relational database management system (RDBMS), Structured Query Language (SQL) is used to interact with the database. Basic retrieval queries involve fetching data from one or more tables based on certain conditions. Here are some common SQL retrieval queries

```
SELECT * FROM TableName;
```

Select Specific Columns from a Table:

```
SELECT Column1, Column2 FROM TableName;
```

Alias for Columns:

```
SELECT Column1 AS Alias1, Column2 AS Alias2 FROM TableName;
```

Filtering Rows with WHERE Clause:

```
SELECT * FROM TableName WHERE Condition;
```

Example:

```
SELECT * FROM Employees WHERE Department = 'IT';
```

Sorting Rows with ORDER BY:

```
SELECT * FROM TableName ORDER BY ColumnName [ASC | DESC];
```

Example:

```
SELECT * FROM Products ORDER BY Price DESC;
```

Insert, Delete, and Update Statements

Data Definition Language (DDL)- CREATE

CREATE It is used to create a new table in the database.

Syntax: CREATE TABLE TABLE_NAME (COLUMN_NAME DATATYPES[,...]);

Example: CREATE TABLE EMPLOYEE(Name VARCHAR2(20), Email VARCHAR2(100), DOB DATE);

Data Manipulation Language

- DML commands are used to modify the database. It is responsible for all form of CHANGES in the database.
- The command of DML is not auto-committed that means it can't permanently save all the changes in the database. They can be rollback. Here are some commands that come under

DML:

INSERT

UPDATE

DELETE

1. Insert Statement:

The INSERT statement is used to add new records to a table. The basic syntax is as follows:

INSERT INTO table_name (column1, column2, column3, ...)

VALUES (value1, value2, value3, ...);

Example:

INSERT INTO employees (employee_id, first_name, last_name, salary)

VALUES (101, 'John', 'Doe', 50000);

This example inserts a new record into the "employees" table with specific values for the columns.

2. Delete Statement:

The DELETE statement is used to remove records from a table based on a condition. The basic syntax is as follows:

```
DELETE FROM table_name
```

```
WHERE condition;
```

Example:

```
DELETE FROM employees
```

```
WHERE employee_id = 101;
```

This example deletes the record from the "employees" table where the employee_id is 101.

3. Update Statement:

The UPDATE statement is used to modify existing records in a table based on a condition. The basic syntax is as follows:

```
UPDATE table_name
```

```
SET column1 = value1, column2 = value2, ...
```

```
WHERE condition;
```

Example:

```
UPDATE employees
```

```
SET salary = 55000
```

```
WHERE employee_id = 101;
```

This example updates the salary column for the employee with the ID 101 in the "employees" table.